Team 12 CIT433027

Risk-Sensitive Domain-Randomized Online Planning for Dexterous Manipulation

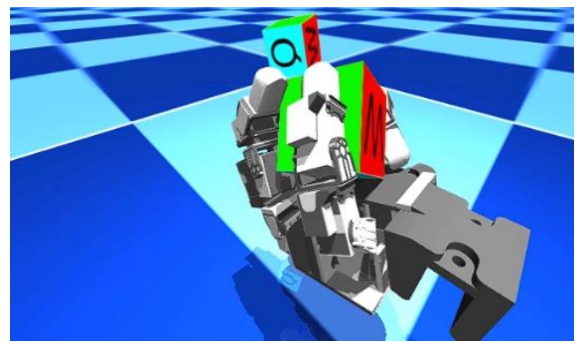
Ishwar Govind
Technical University of Munich
Ishwar.govind@tum.de

Motivation

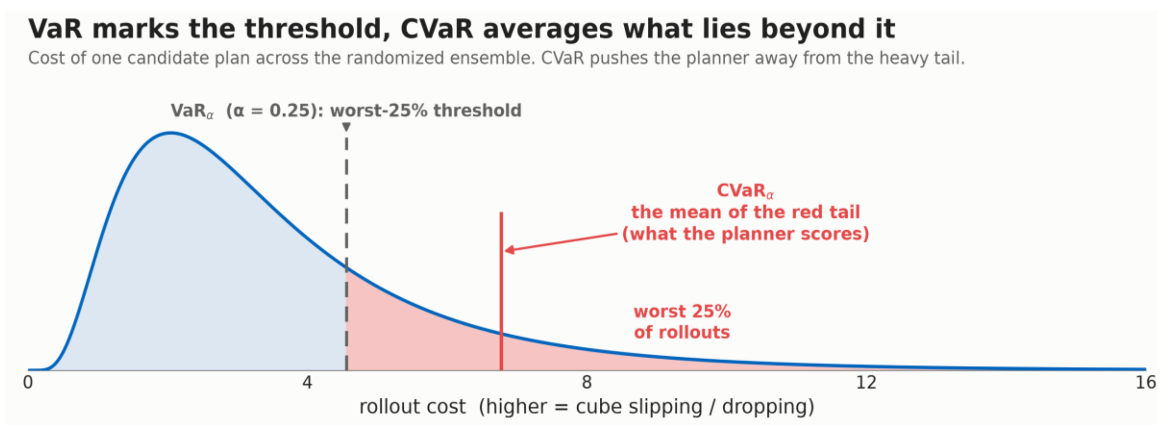
- Sampling-based MPC adapts online with no offline training, but it plans in a single nominal simulator.
- In contact-rich manipulation, small differences in friction, actuator gain or damping change which contacts occur.
- The planner then ends up exploiting dynamics the real system doesn't actually have.

Problem Setup

- LEAP hand, in-hand cube reorientation (MuJoCo MPC, CPU).
- Planner model ≠ executed model: **friction, actuator gain, damping, frictionloss and Kv perturbed** at std-multiplier ∈ {0, 1, 2}
- Baselines: Predictive Sampling (PurePS) and CEM (PureCEM).

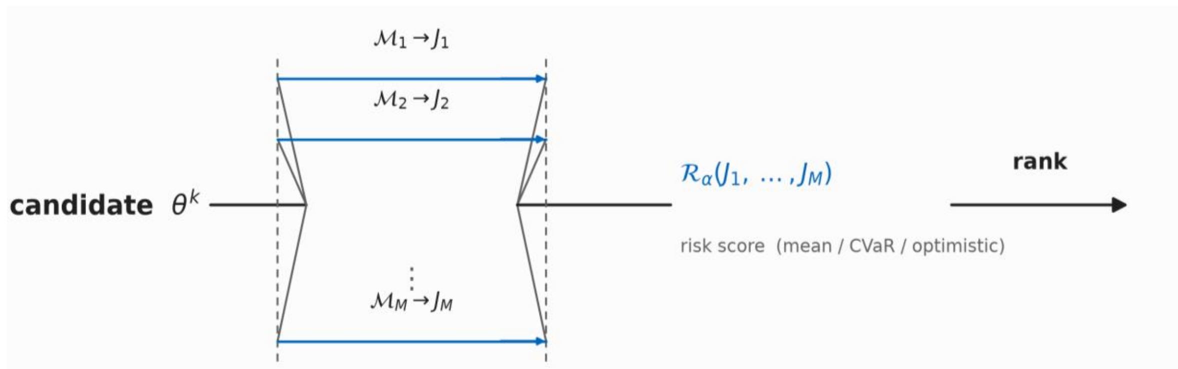


LEAP hand reorienting the cube toward the goal pose



Contributions

- **M MuJoCo models** built once at startup, each with its physics perturbed and then held fixed for the whole episode.
- **Wrapper Planner** adds the ensemble and risk scoring over any ranking-based planner, driving both Predictive Sampling and CEM.
- **Risk Operator** turns each candidate's M rollout costs into one ranking score: CVaR - optimistic, mean or pessimistic based on value of α.
- **Hierarchical picking:** the inner planner screens on the nominal model, then re-ranks the top few across the ensemble.



Method: Risk-Aware DR-CEM

Algorithm 1 Risk-aware DR-CEM (one replanning step)

Require: ensemble $\{\mathcal{M}_j\}_{j=1}^{M}$ (fixed); N samples, K candidates, $n_e < K$ elites, risk level α

- $\bar{\theta} \leftarrow \textsc{Shift}(\theta)$ ▷ receding horizon
- $\theta^i \leftarrow \bar{\theta} + \varepsilon^i,\ \ \varepsilon^i \sim \mathcal{N}(0, \sigma^2),\ \ i = 1, \dots, N$ ▷ sample plans
- $\mathcal{C} \leftarrow K$ best under $J(\theta^i; \mathcal{M}_0)$ ▷ screen on nominal model
- $S_k \leftarrow \mathcal{R}_\alpha\big(\{J(\theta^k; \mathcal{M}_j)\}_{j=1}^{M}\big),\ \ k \in \mathcal{C}$ ▷ M rollouts each: risk score
- $\mathcal{E} \leftarrow n_e$ best under S_k ▷ elites, ranked by risk
- $\theta \leftarrow \sum_{k\in\mathcal{E}} w_k \theta^k,\quad \sigma^2 \leftarrow \sum_{k\in\mathcal{E}} w_k (\theta^k - \theta)^{\odot 2}$ ▷ CEM update
- **execute** π_θ until next replan

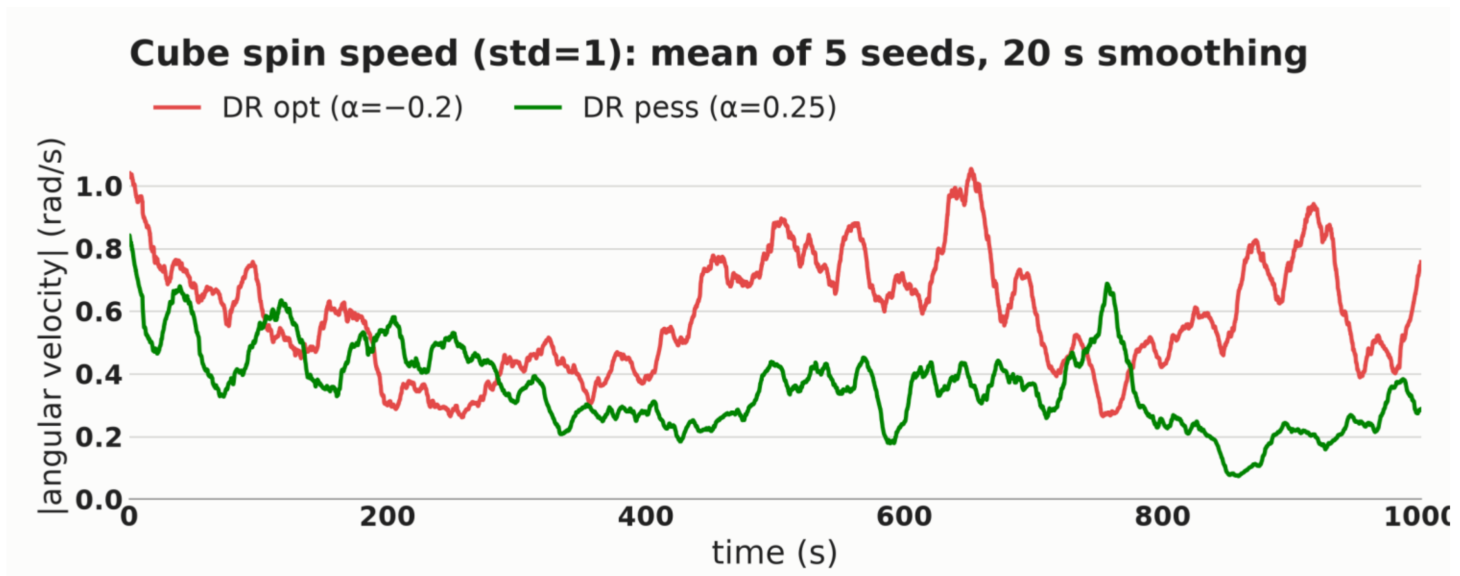
Risk operator. $\mathcal{R}_\alpha$ averages the worst $\lceil \alpha M \rceil$ costs ($0 < \alpha < 1$, pessimistic), all M ($\alpha = 1$, mean), or the best $\lceil |\alpha| M \rceil$ ($\alpha < 0$, optimistic). **Cost:** $N + KM$ rollouts per step.

- Screen: try N noisy plans on the nominal model, keep the K best.
- Stress: replay each survivor on all M randomized models, then combine its M costs with the risk operator Rα.
- Update: CEM elite mean and variance, ranked by risk score.
- Budget: N + K·M rollouts per replan (about 2N when K = N/M).

Risk Operator

- α = 1: mean cost over the ensemble (risk-neutral).
- 0 < α < 1: mean of the worst ⌈αM⌉ costs (CVaR, cautious).
- α < 0: mean of the best ⌈|α|M⌉ costs (optimistic, aggressive).

Optimistic vs. Pessimistic Spin (std = 1)

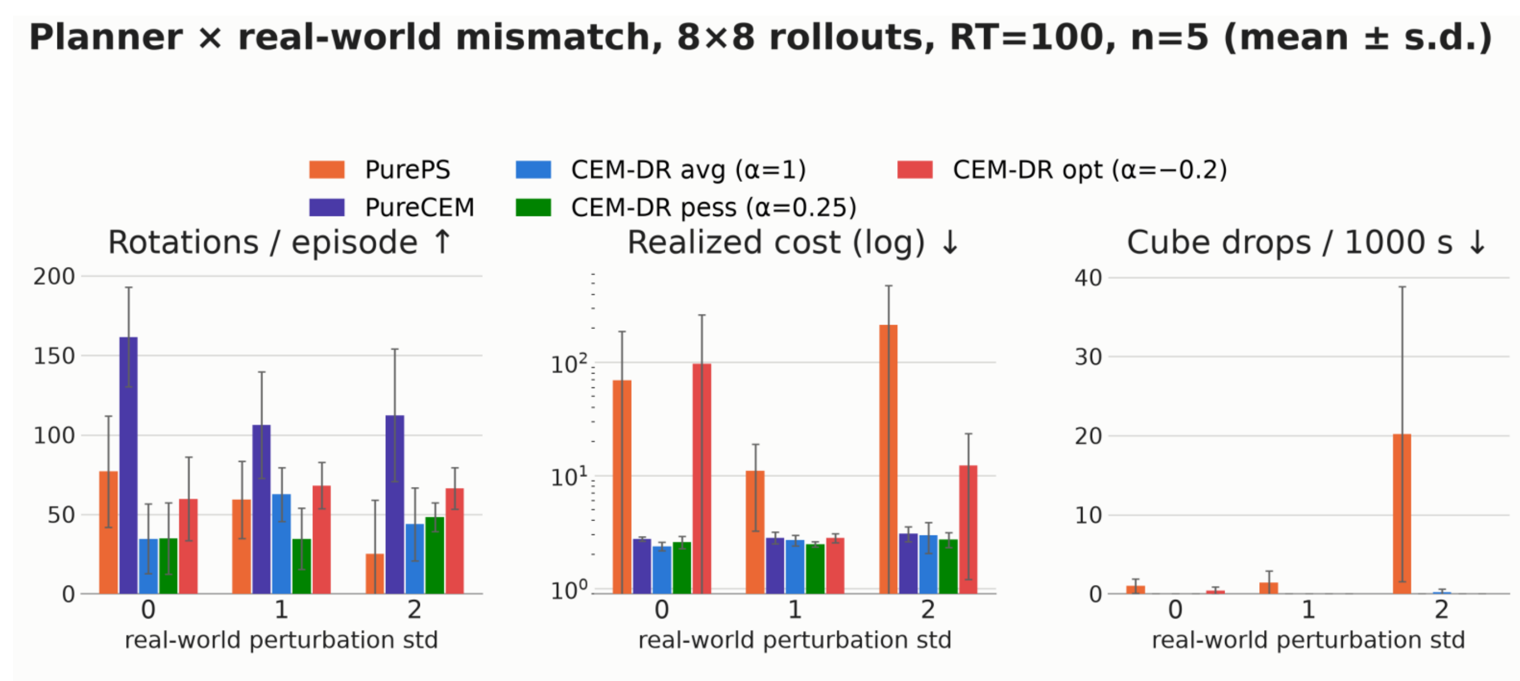


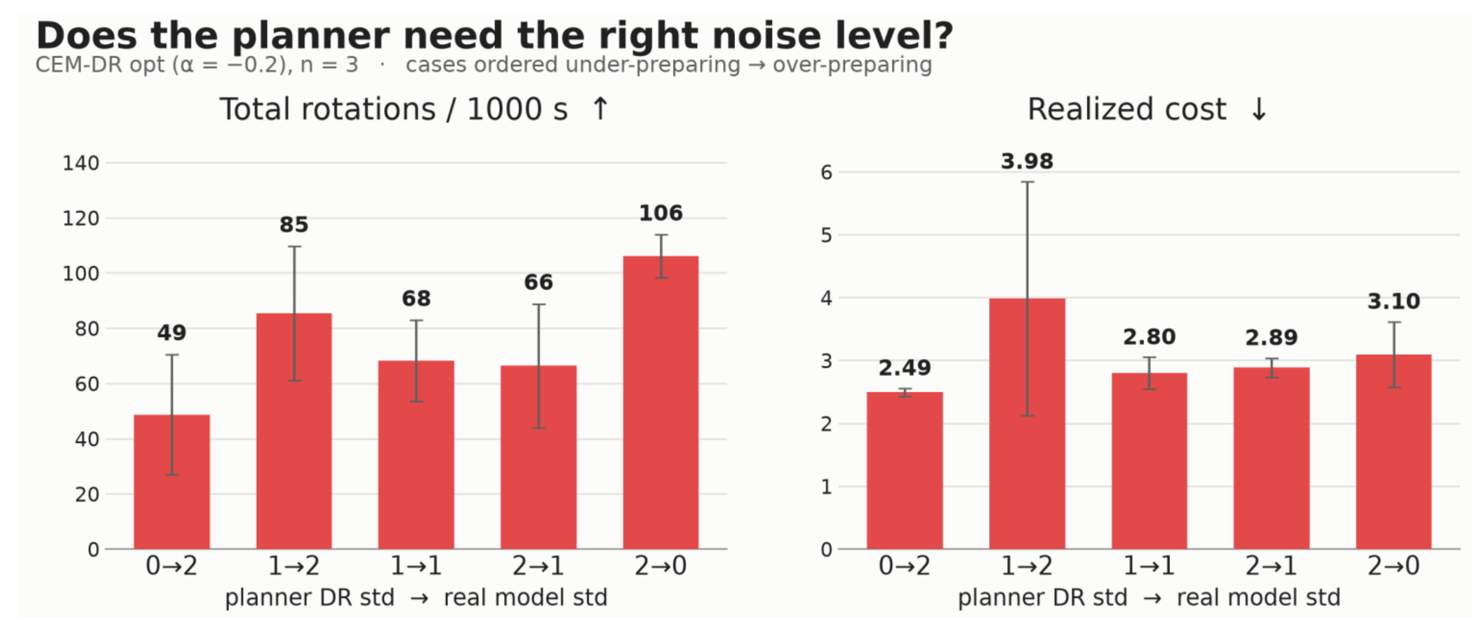
DR opt sustains a higher spin speed than DR pess on average

Experiments

- 5 planners × 3 real-world perturbation levels, n = 5 seeds, 1000 s episodes.
- Paired seeds: identical goal sequence and DR draw across risk operators.
- Matched budget: M = 8 models × K = 8 candidates = 64 DR rollouts, RT = 100 (% of real-time; 100 = true real-time).
- Also: a planner-DR-std vs real-std mismatch sweep (n = 3), and a rollout-budget probe at slowed sim-time (RT = 12).

Results





Conclusions

- Plain CEM is already drop-free at every std, so DR only really pays off where the baseline fails.
- Predictive Sampling falls apart under mismatch (20 drops, cost 214 at std = 2), but no CEM-DR variant ever does.
- When noise mismatches, more DR helps only when the real world is harder than expected (1→2 > 0→2). Over-preparing past the real difficulty buys nothing (2→1 ≈ 1→1)
- On CPU the Hz dominates: running 16× the rollouts at matched hertz turns 0 drops into 18.7 for optimistic.
- Early 16-core runs looked like DR failing outright, moving to 112 cores fixed it. (planner and physics threads run without conditional variables, unblocked).